
```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@marketplace-s-1vcpu-2gb-blr1:~# sudo certbot --nginx -d 167.71.235.72.sslip.io
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Enter email address (used for urgent renewal and security notices)
(Enter 'c' to cancel): bosemrgn@gmail.com

-----
Please read the Terms of Service at
https://letsencrypt.org/documents/LE-SA-v1.6-August-18-2025.pdf. You must agree
in order to register with the ACME server. Do you agree?
-----
(Y)es/(N)o: y

-----
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
-----
(Y)es/(N)o: y
Account registered.
Requesting a certificate for 167.71.235.72.sslip.io

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem
Key is saved at: /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem
This certificate expires on 2026-08-03.
These files will be updated when the certificate renews.

```

```

GNU nano 6.2
# HTTP server
server {
    listen 80;
    listen [::]:80;

    server_name 167.71.235.72.sslip.io;

    location / {
        root /var/www/html;
        index index.html;
    }
}

# HTTPS server
server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name 167.71.235.72.sslip.io;

    root /var/www/html;
    index index.html;

    # Certificate (THIS WAS MISSING)
    ssl_certificate /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem;
}

```

Website Identity
 Website: 167.71.235.72.sslip.io
 Owner: This website does not supply ownership information.
 Verified by: Let's Encrypt [View Certificate](#)

Privacy & History
 Have I visited this website prior to today? No
 Is this website storing information on my computer? Yes, cookies [Clear Cookies and Site Data](#)
 Have I saved any passwords for this website? Yes [View Saved Passwords](#)

Technical Details
 Connection Encrypted (TLS_AES_256_GCM_SHA384, 256 bit keys, TLS 1.3)
 The page you are viewing was encrypted before being transmitted over the Internet.
 Encryption makes it difficult for unauthorized people to view information traveling between computers. It is therefore unlikely that anyone read this page as it traveled across the network.
 This website complies with the Certificate Transparency policy. [Help](#)

167.71.235.72.sslip.io
R12
ISRG Root X1

Subject Name
 Common Name 167.71.235.72.sslip.io

Issuer Name
 Country US
 Organization Let's Encrypt
 Common Name [R12](#)

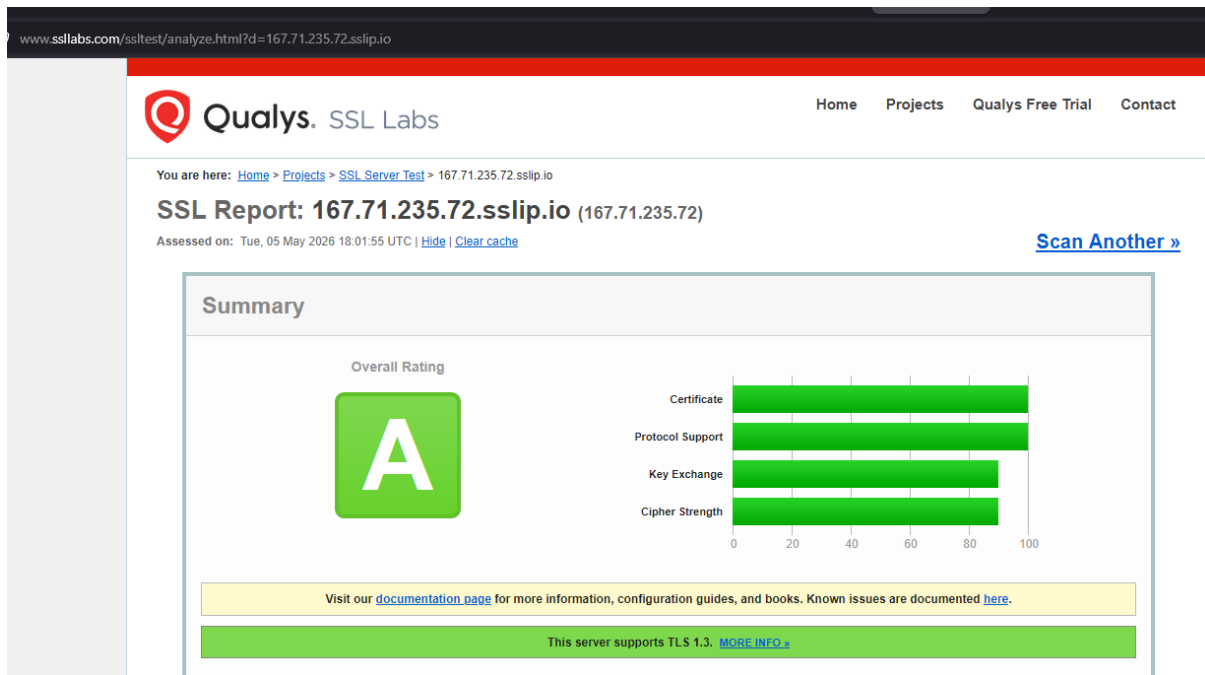
Validity
 Not Before Tue, 05 May 2026 16:52:23 GMT
 Not After Mon, 03 Aug 2026 16:52:22 GMT

Subject Alt Names
 DNS Name 167.71.235.72.sslip.io

Public Key Info
 Algorithm RSA
 Key Size 2048
 Exponent 65537
 Modulus DF:70:7A:FE:BD:DD:CF:8D:CB:DE:09:71:9F:61:2C:76:C0:0D:69:D2:A7:F5:4E:77:3A:...

Miscellaneous
 Serial Number 05:A9:22:D0:50:FF:50:21:EA:33:71:E2:A5:B8:C7:E6:16:31
 Signature Algorithm SHA-256 with RSA Encryption
 Version 3
 Download [PEM \(cert\)](#) [PEM \(chain\)](#)

Purposes	Server Authentication
Subject Key ID	
Key ID	1F:A1:7F:8B:33:B3:C0:84:45:ED:3D:69:E3:E7:1F:C3:C7:EF:4C:E8
Authority Key ID	
Key ID	00:B5:29:F2:2D:8E:6F:31:E8:9B:4C:AD:78:3E:FA:DC:E9:0C:D1:D2
CRL Endpoints	
Distribution Point	http://r12.c.lencr.org/50.crl
Authority Info (AIA)	
Location	http://r12.i.lencr.org/
Method	CA Issuers
Certificate Policies	
Policy	Certificate Type (2.23.140.1.2.1)
Value	Domain Validation
Embedded SCTs	
Log Name	Sectigo 'Tiger2026h2'
Log ID	C8:A3:C4:7F:C7:B3:AD:B9:35:6B:01:3F:6A:7A:12:6D:E3:3A:4E:43:A5:C6:46:F9:97:...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Tue, 05 May 2026 17:50:53 GMT
Log Name	IPng Networks 'Gouda2026h2'
Log ID	1A:8B:9D:6B:0F:FE:BF:81:B4:79:39:C6:D2:31:0A:86:D6:D1:02:D4:F0:46:E2:18:2C:9...
Signature Algorithm	SHA-256 ECDSA
Version	1
Timestamp	Tue, 05 May 2026 17:50:54 GMT



For A+ I have changed:

```
root@marketplace-s-1vcpu-2gb-blr1:~# sudo systemctl restart nginx
root@marketplace-s-1vcpu-2gb-blr1:~# sudo cat /etc/nginx/sites-available/default
#
# HTTP to HTTPS Redirect
#
server {
    listen 80;
    listen [::]:80;

    server_name 167.71.235.72.sslip.io;

    return 301 https://$host$request_uri;
}

#
# HTTPS Server (A+ Config)
#
server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;

    server_name 167.71.235.72.sslip.io;

    root /var/www/html;
    index index.html index.htm;

    # Let's Encrypt Certificate
    ssl_certificate /etc/letsencrypt/live/167.71.235.72.sslip.io/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/167.71.235.72.sslip.io/privkey.pem;

    # TLS Protocols
    ssl_protocols TLSv1.2 TLSv1.3;

    # Strong Ciphers
    ssl_prefer_server_ciphers on;
    ssl_ciphers ECDH+AESGCM:EDH+AESGCM;

    # Session Settings
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1d;
    ssl_session_tickets off;

    # HSTS (needed for A+)
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;

    # OCSP Stapling (fixed)
    ssl_stapling on;
    ssl_stapling_verify on;

    # Resolver (fixes warning)
    resolver 8.8.8.8 1.1.1.1 valid=300s;
    resolver_timeout 5s;

    # Website content
    location / {
        try_files $uri $uri/ =404;
    }
}
```

You are here: [Home](#) > [Projects](#) > [SSL Server Test](#) > 167.71.235.72.sslip.io

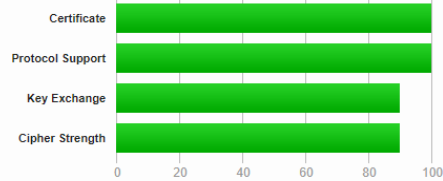
SSL Report: 167.71.235.72.sslip.io (167.71.235.72)

Assessed on: Tue, 05 May 2026 18:22:42 UTC | [Hide](#) | [Clear cache](#)

[Scan Another »](#)

Summary

Overall Rating



Visit our [documentation page](#) for more information, configuration guides, and books. Known issues are documented [here](#).

This server supports TLS 1.3. [MORE INFO »](#)

HTTP Strict Transport Security (HSTS) with long duration deployed on this server. [MORE INFO »](#)

POODLE (SSLV3)	No, SSL 3 not supported (more info)
POODLE (TLS)	No (more info)
Zombie POODLE	No (more info)
GOLDENDOODLE	No (more info)
OpenSSL 0-Length	No (more info)
Sleeping POODLE	No (more info)
Downgrade attack prevention	Yes, TLS_FALLBACK_SCSV supported (more info)
SSL/TLS compression	No
RC4	No
Heartbeat (extension)	No
Heartbleed (vulnerability)	No (more info)
Ticketbleed (vulnerability)	No (more info)
OpenSSL CCS vuln. (CVE-2014-0224)	No (more info)
OpenSSL Padding Oracle vuln. (CVE-2016-2107)	No (more info)
ROBOT (vulnerability)	No (more info)
Forward Secrecy	Yes (with most browsers) ROBUST (more info)
ALPN	Yes h2 http/1.1
NPN	Yes h2 http/1.1
Session resumption (caching)	Yes
Session resumption (tickets)	No
OCSP stapling	No
Strict Transport Security (HSTS)	Yes max-age=63072000; includeSubDomains; preload
HSTS Preloading	Not in: Chrome Edge Firefox IE
Public Key Pinning (HPKP)	No (more info)
Public Key Pinning Report-Only	No
Public Key Pinning (Static)	No (more info)
Long handshake intolerance	No
TLS extension intolerance	No
TLS version intolerance	No
Incorrect SNI alerts	No
Uses common DH primes	No, DHE suites not supported
DH public server param (Ys) reuse	No, DHE suites not supported
ECDH public server param reuse	No

```

root@marketplace-s-1vcpu-2gb-blr1:~# openssl s_client -connect 167.71.235.72:443 -servername 167.71.235.72
CONNECTED(00000003)
depth=2 C = US, 0 = Internet Security Research Group, CN = ISRG Root X1
verify return:1
depth=1 C = US, 0 = Let's Encrypt, CN = R12
verify return:1
depth=0 CN = 167.71.235.72:sslip.io
verify return:1
---
Certificate chain
 0 s:CN = 167.71.235.72:sslip.io
  i:C = US, 0 = Let's Encrypt, CN = R12
  a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: May  5 16:52:23 2026 GMT; NotAfter: Aug  3 16:52:22 2026 GMT
 1 s:C = US, 0 = Let's Encrypt, CN = R12
  i:C = US, 0 = Internet Security Research Group, CN = ISRG Root X1
  a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Mar 13 00:00:00 2024 GMT; NotAfter: Mar 12 23:59:59 2027 GMT
---
Server certificate
-----BEGIN CERTIFICATE-----
MIIFBjCCAA+6gAwIBAgISBaki0FD/UCHqM3Hipb.jH5hYxMA0GCSqGSIb3DQEBCwUA
MDMxCzAJBgNVBAYTA1VTMRYwFAYDVQQKEw1MZXQncyBFbmNyeXB0MQwwCgYDVQDD
EwNSMTIwHhcNMjYwOTA1MTY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1
ExYXNjc3RlbnNzZm1lLjcyLnNzbGlwLm1vMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8A
MIIBCAQEA38B6/r3dz43L3glxn2EsdsANadKn9U530uhUTH1KufKxSn609W+B
QSiXp16cLP7095NeJis9MzyE+O/4rrRe/eleEpRqAu72uaiqDu5X8EgltAAawiOw
PFH00aRpF0bGQJsvf3TQ1Q2MmGLjJKJshSF4e9K0MG0fw4fCn1I94eQCKDEHq1
Nof7rviTBoSsvH+HJb8yaaJFCUT0NXZ3aU3dJ/K1mnQ7ycqbJBev5cDkKiASHCBF
QTLPKdGcQtAai+VX1TnJs120qxIdj2VoeUEFIWHPV2GpCJSOCWKKPqCwLnSvHAl
Om1CnbnvVJIXdTF4k9rDlnkbtXJ3PU31wIDAQABO4ICJDCCAIAwDgYDVVR0PAQH/
BAQDAgWgMBMGA1UdJQQMMAoGCCsGAQUFBwMBMAwGA1UdEwEB/wQCMAAwHQYDVVR0O
BBYEFB+hf4szs8CERe09aePnH8PH70zoMB8GA1UdIwQYMBaAFAC1Kf1tjm8x6JtM
rXg++tZpDNHSMDCGCSGAUQFBwEBBCCwJTAjBqqrBgEFBQcwAoYXaHR0cDovL3Iz
MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1MjY1
bzATBgNVHSAEDDAKAgQ8meBDAECAu8qNVHR8EJzA1MCOgIaAfhh1odHRwOi8v
c3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1v
c3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1vY3RlbnNzZm1vLm1v
Ami.jxH/Hs625NwsBP2p6Em3j0k5DpcZG+ZetOXW2Hc+AAAAbnf1Dp9QAAQDAEw
RQIQAQgJr9iQNTL2d4yE1Nwik29XKs28I45zILxMvbWEECIQDRw/Zhek/SRTiw
IYs5SWjDRp/7s87Z0o6Q/dU0TSmusPwB+ABqLnWSP/r+BtHk5xtIxCobW0QLU8Ebi
GCYd419eJiXvAAA8bnf1Dqu0ACAAABQAPSdhJBAMARzBFaEAr/JMTVuY5CCFH0dW
PtBqvcSacl/zm0RPMEOUf2N1mwECIHhGA8x00eqm5yotUvavCmrB5+890I6hrsXj
B8LTmqWuMA0GCSqGSIb3DQEBCwUAA4IBAQCfrdJ9Zn4ZU7M2XESTAJUhrLn1f7R
sG0cQn8vrpERUEBA935qpfn3EmSzT+8eVcrh2rsviNvjbXIp64pL+/78106hVZyr
YQqR5e64REtRzsAJluTzV1bIvU6pcVFDUmlvnufyUHSX30Cw05SLakWNZA4nWS
oZLHJB+7oHaffmVmECdUY1Czf9Kfek3CK035NPDcZrOQU3SxgvRISmKiHr76LCK
mGyTpY+63G0CkXS4KI0KkyCvFAxRe2NZY6+Fv3RxlUGxxiydwmX0SNq2Pux2zt
OdvBZ0ZrQRzTckYAhRAXchohq2S7gEHXtL8Mko/xSnohL1HfUCHc7+7c
-----END CERTIFICATE-----
subject=CN = 167.71.235.72:sslip.io
issuer=C = US, 0 = Let's Encrypt, CN = R12
---
No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 3145 bytes and written 404 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---

```

Conclusion

In this experiment, a secure web server was successfully configured using Nginx on a cloud-based Ubuntu system. A trusted SSL/TLS certificate was obtained using Let's Encrypt and integrated into the server to enable HTTPS communication. Initially, the server operated with default configurations, resulting in a lower SSL rating. By applying strong security measures such as enforcing TLS 1.2 and TLS 1.3 protocols, configuring secure cipher suites, enabling HTTP to HTTPS redirection, and implementing HTTP Strict Transport Security (HSTS), the overall security posture of the server was significantly improved.

The final configuration was validated using SSL Labs, where the server achieved an A+ rating, demonstrating compliance with modern web security standards. This experiment provided practical understanding of SSL/TLS implementation, certificate management, and secure server configuration, which are essential for building and maintaining secure web applications.

- 2) Install DVWA or Juice Shop. Place ModSecurity WAF in front of it with OWASP Core Rule Set. Verify it blocks SQLi and XSS.

DVWA Security

Security Level

Security level is currently: **high**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

High

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

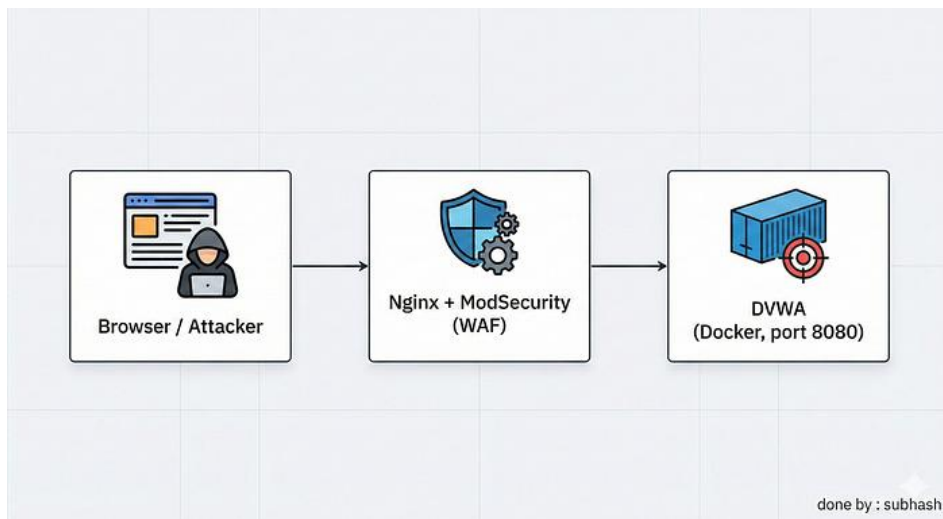
You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Username: admin
Security Level: high
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 "Development"



```

subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx$ docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
0aff42b7d66d   vulnerables/web-dvwa   "/main.sh"              4 hours ago   Up 4 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   dvwa-security-lab_dvwa_1
c9b398241895   mysql:5.7             "docker-entrypoint.s..." 4 hours ago   Up 4 hours   3306/tcp, 33060/tcp                  dvwa-security-lab_db_1
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx$

```

```

subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:~/dvwa-security-lab$ cat docker-compose.yml
version: '3'

services:
  dvwa:
    image: vulnerables/web-dvwa
    ports:
      - "8080:80"
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
    depends_on:
      - db

  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:~/dvwa-security-lab$

```



Username

Password

Login



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: **high**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. Its use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

High

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Username: admin
Security Level: high
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 "Development"

Step 1 : Attacking DVWA without a WAF (Reality Check)

Before even touching ModSecurity, I wanted to see **how bad things actually are** when an app runs *without* a Web Application Firewall.

So I deployed **DVWA directly on Docker (port 8080)** and tried a few classic attacks.

What I tested:

- SQL Injection
- Command Injection
- XSS (Reflected & Stored)

The screenshot shows the DVWA web application interface. At the top is the DVWA logo. On the left is a sidebar menu with various security challenges. The main content area is titled "Vulnerability: SQL Injection". It features a "User ID:" input field and a "Submit" button. Below the input field, there are five rows of red text showing the results of SQL injection attacks using the payload "ID: ' OR '1' ='1". The results show the first name and surname of the user whose ID was injected. The "More Information" section lists several links to external resources about SQL injection. At the bottom, there is a footer with the text "Damn Vulnerable Web Application (DVWA) v1.10 *Development*".

DVWA

Vulnerability: SQL Injection

User ID:

ID: ' OR '1' ='1
First name: admin
Surname: admin

ID: ' OR '1' ='1
First name: Gordon
Surname: Brown

ID: ' OR '1' ='1
First name: Hack
Surname: Me

ID: ' OR '1' ='1
First name: Pablo
Surname: Picasso

ID: ' OR '1' ='1
First name: Bob
Surname: Smith

More Information

- <http://www.securiteam.com/securityreviews/SDP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-okul/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 *Development*



Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript

DVWA Security
PHP Info
About

Logout

Username: admin
Security Level: low
PHPIDS: disabled

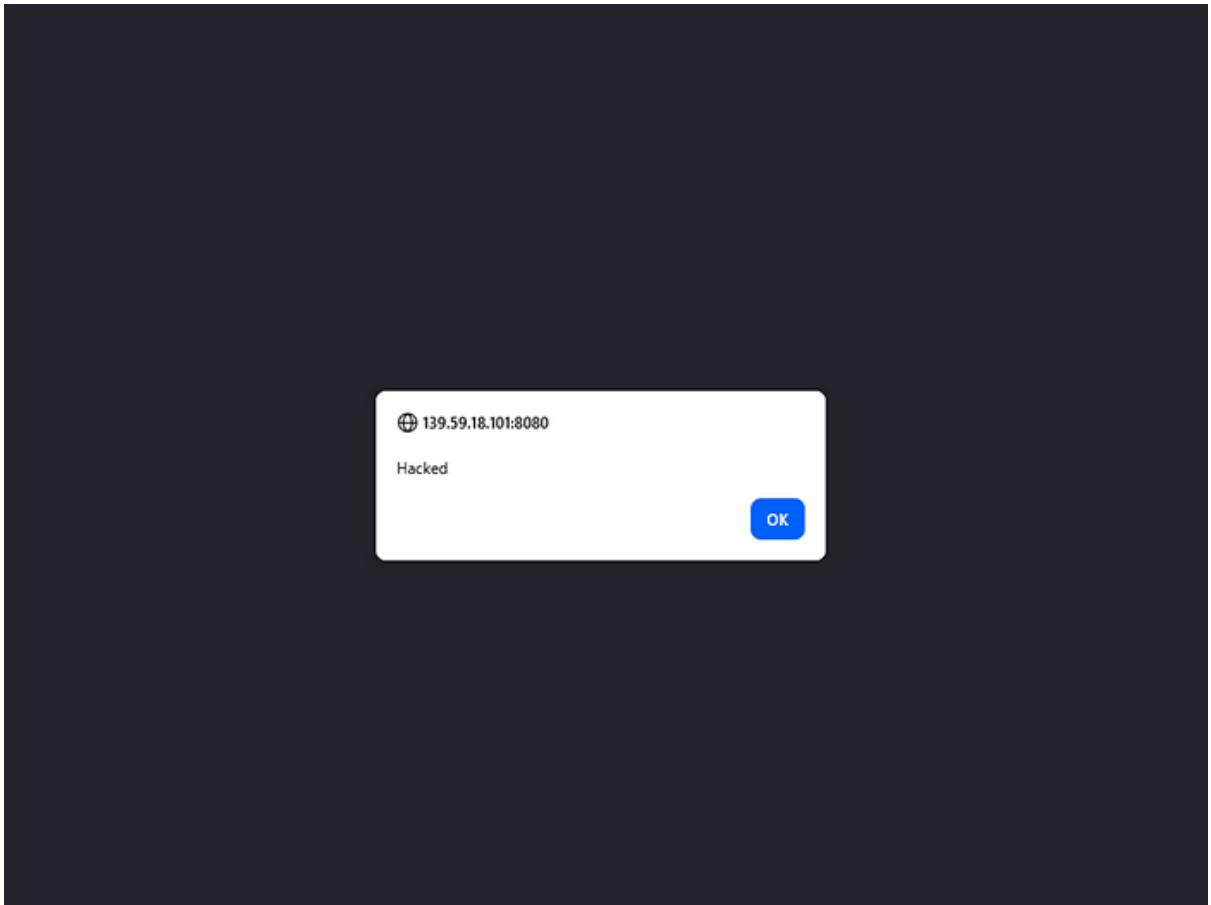
Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

Damn Vulnerable Web Application (DVWA) v1.10 "Development"





- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection**
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.074 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.063 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.067 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.063/0.068/0.074/0.000 ms
CHANGELOG.md
COPYING.txt
README.md
about.php
config
docs
dvwa
external
favicon.ico
hackable
ids_log.php
index.php
instructions.php
login.php
logout.php
php.ini
phpinfo.php
robots.txt
security.php
setup.php
vulnerabilities
```

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_Injection

This step was important because:

- I needed a **baseline**
- Otherwise there's no proof that ModSecurity actually *did* something later

Once I confirmed attacks were working freely, **then only** I moved to installing a WAF.

Installing ModSecurity with Nginx.

```
objjs/addon/src/nginx_http_modsecurity_header_filter.o \
objjs/addon/src/nginx_http_modsecurity_body_filter.o \
objjs/addon/src/nginx_http_modsecurity_log.o \
objjs/nginx_http_modsecurity_module_modules.o \
-Wl,-rpath,/usr/local/modsecurity/lib -L/usr/local/modsecurity/lib -lmodsecurity \
-shared
make[1]: Leaving directory '/usr/local/src/nginx-1.18.0'
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/usr/local/src/nginx-1.18.0$ ls objjs | grep modsecurity
nginx_http_modsecurity_module.so
nginx_http_modsecurity_module_modules.c
nginx_http_modsecurity_module_modules.o
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/usr/local/src/nginx-1.18.0$ sudo mkdir -p /etc/nginx/modules
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/usr/local/src/nginx-1.18.0$ sudo cp objjs/nginx_http_modsecurity_module.so /etc/nginx/modules/
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/usr/local/src/nginx-1.18.0$ cd /etc/nginx/
conf.d/ modules/ modules-available/ modules-enabled/ sites-available/ sites-enabled/ snippets/
```

Initially, ModSecurity was enabled globally inside the http {} block.

However, this applies WAF rules to every hosted application, which is not ideal.

To follow a more realistic and controlled approach, I enabled ModSecurity only at the virtual host level for DVWA.

This allows targeted protection, cleaner debugging, and mirrors real-world WAF deployments.

Why?

- Cleaner setup
- Real-world approach
- No unnecessary blocking elsewhere

```
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$ ls
dvwa
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$ cat dvwa
server {
    listen 80;
    server_name _;

    # Enable ModSecurity
    modsecurity on;
    modsecurity_rules_file /etc/nginx/modsecurity.conf;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
subhash@ubuntu-s-2vcpu-8gb-160gb-intel-blr1-01:/etc/nginx/sites-enabled$ |
[0] 0: bash*
```

Now the waf is in front of DVWA.

Writing Custom WAF Rules

Instead of using CRS directly, I wrote **minimal custom rules** to clearly prove blocking behavior.

XSS Attempt:



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

testuser

Message *

Sign Guestbook

Clear Guestbook

Name: test

Message: This is a test comment.

Name: testuser

Message: alert(1)

Name: testuser

Message: alert(1)

Name: testuser

Message:

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

View Source

View Help

Username: admin

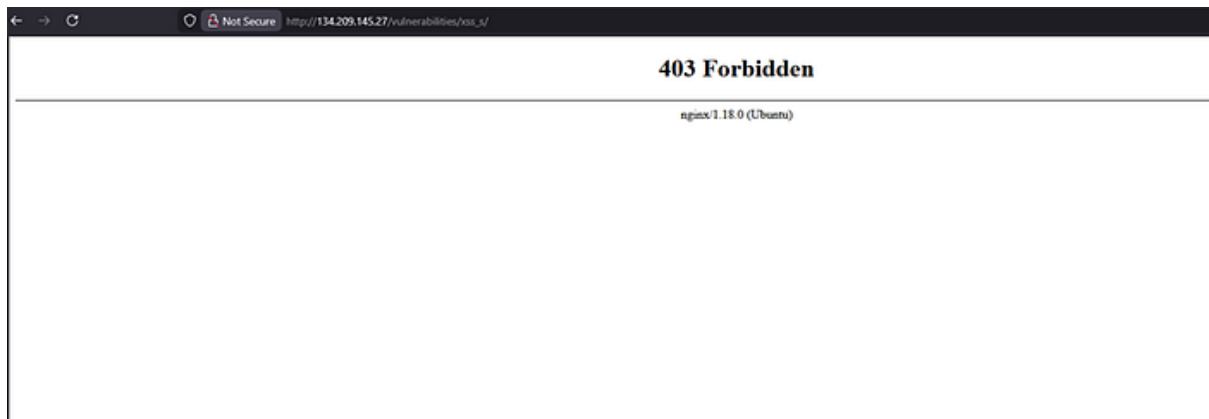
Security Level: high

PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.10 "Development"

```
[{"id":1,"type":"error","message":"load-alert() against variable 'ARGS[name]' (Value: <script>alert(1)</script>) [file '/etc/nginx/modsecurity.conf'] [line '289'] [id '1003'] [rev ''] [msg 'XSS Attack Detected'] [data ''] [severity '0'] [ver ''] [maturity '0'] [accuracy '0'] [hostname '134.209.145.27'] [uri '/vulnerabilities/xss_r'] [unique_id '176694473267.131606'] [ref '00,700,703,25'] , client: 157.51.67.85, server: , request: 'GET /vulnerabilities/xss_r/?name=X&script=alert(1)&script=alert(1)&script=alert(1) HTTP/1.1', host: '134.209.145.27', referer: 'http://134.209.145.27/vulnerabilities/xss_r/'"}, {"id":2,"type":"error","message":"load-alert() against variable 'ARGS[name]' (Value: <script>alert(1)</script>) [file '/etc/nginx/modsecurity.conf'] [line '289'] [id '1003'] [rev ''] [msg 'XSS Attack Detected'] [data ''] [severity '0'] [ver ''] [maturity '0'] [accuracy '0'] [hostname '134.209.145.27'] [uri '/vulnerabilities/xss_r'] [unique_id '176694473267.131606'] [ref '00,700,703,25'] , client: 157.51.67.85, server: , request: 'GET /vulnerabilities/xss_r/?name=X&script=alert(1)&script=alert(1)&script=alert(1) HTTP/1.1', host: '134.209.145.27', referer: 'http://134.209.145.27/vulnerabilities/xss_r/'"}, {"id":3,"type":"error","message":"load-alert() against variable 'ARGS[name]' (Value: <script>alert(1)</script>) [file '/etc/nginx/modsecurity.conf'] [line '289'] [id '1003'] [rev ''] [msg 'XSS Attack Detected'] [data ''] [severity '0'] [ver ''] [maturity '0'] [accuracy '0'] [hostname '134.209.145.27'] [uri '/vulnerabilities/xss_r'] [unique_id '176694473267.131606'] [ref '00,700,703,25'] , client: 157.51.67.85, server: , request: 'GET /vulnerabilities/xss_r/?name=X&script=alert(1)&script=alert(1)&script=alert(1) HTTP/1.1', host: '134.209.145.27', referer: 'http://134.209.145.27/vulnerabilities/xss_r/'"}, {"id":4,"type":"error","message":"load-alert() against variable 'ARGS[name]' (Value: <img src=x onerror=alert(1)>) [file '/etc/nginx/modsecurity.conf'] [line '289'] [id '1003'] [rev ''] [msg 'XSS Attack Detected'] [data ''] [severity '0'] [ver ''] [maturity '0'] [accuracy '0'] [hostname '134.209.145.27'] [uri '/vulnerabilities/xss_r'] [unique_id '176694473267.131606'] [ref '00,700,703,25'] , client: 157.51.67.85, server: , request: 'GET /vulnerabilities/xss_r/?name=X&script=alert(1)&script=alert(1)&script=alert(1) HTTP/1.1', host: '134.209.145.27', referer: 'http://134.209.145.27/vulnerabilities/xss_r/'"}]
```

[19]



Conclusion:

When ModSecurity was introduced as a Web Application Firewall in front of DVWA, the story changed. The same attacks that worked earlier were now **blocked, logged, and controlled**. Seeing 403 responses instead of successful exploitation made it clear how powerful a properly configured WAF can be.